

CSE 5095: Types and Programming Languages

Type Safety

Stefan Muller

Thursday, Sept. 10

1 Type Safety

Last time, we saw syntactically valid expressions that don't step. For example, $\bar{1} + \text{"Hello"}$. We say such expressions are “stuck.” There are also valid expressions that will step, but will (eventually) step to expressions that can no longer step (these will “get stuck”). For example, $(\bar{1} + \bar{2}) + \text{"Hello"} \mapsto \bar{3} + \text{"Hello"}$. We designed a type system to prevent these errors and argued informally that a well-typed program shouldn't get stuck like this.

The slogan of type safety, attributed to Robin Milner, is “well-typed programs can't go wrong.” Type safety is the property that guarantees this: you can see it as a proof that your type system achieves the goal it sets out to. In the context described above (and in most practical type systems), “go wrong” is understood to mean “get stuck” or encounter a run-time type error. We typically prove type safety as two theorems:

- **Progress** says that if e is well-typed, it's a value (i.e., done evaluating) or can take a step. In other words, it's not stuck.
- **Preservation** says that if a well-typed expression takes a step, the new expression is well-typed (with the same type).

Together, these show that a well-typed program won't *become* stuck. For example, if $e_1 : \tau$ and, by progress, $e_1 \mapsto e_2$, then, by preservation, $e_2 : \tau$, so by progress, $e_2 \mapsto e_3$, and so on.

Theorem 1 (Preservation). *If $e : \tau$ and $e \mapsto e'$, then $e' : \tau$.*

Proof. By induction on the derivation of $e \mapsto e'$.

- **STEPADD.** Then $e = \bar{n}_1 + \bar{n}_2$ and $e' = \overline{n_1 + n_2}$. By inversion on TYPEADD, we have $\tau = \text{int}$.¹ By TYPENUM, $e' : \text{int}$.
- **STEPSCAT.** Then $e = \text{"s}_1" \wedge \text{"s}_2"$ and $e' = \text{"s}_1\text{s}_2"$. By inversion on TYPECAT, $\tau = \text{string}$. By TYPESTRING, $e' : \text{string}$.
- **STEPLEN.** Then $e = |\text{"s"}|$ and $e' = |\overline{s}|$. By inversion on TYPELEN, $\tau = \text{int}$. By TYPENUM, $e' : \text{int}$.
- **STEPSEARCHADDDLEFT.** Then $e = e_1 + e_2$ and $e' = e'_1 + e_2$ and $e_1 \mapsto e'_1$. By inversion on TYPEADD, $\tau = \text{int}$ and $e_1 : \text{int}$ and $e_2 : \text{int}$. By induction, $e'_1 : \text{int}$. By TYPEADD, $e' : \text{int}$.
- **STEPSEARCHADDRIGHT.** Then $e = \bar{n}_1 + e_2$ and $e' = \bar{n}_1 + e'_2$ and $e_2 \mapsto e'_2$. By inversion on TYPEADD, $\tau = \text{int}$ and $\bar{n}_1 : \text{int}$ and $e_2 : \text{int}$. By induction, $e'_2 : \text{int}$. By TYPEADD, $e' : \text{int}$.
- **STEPSEARCHCATLEFT.** Basically the same as the case for STEPSEARCHADDDLEFT.

¹This is a proof technique we haven't seen before. Inversion means using the inference rules “upside down.” We want to know what τ is, so we look at the typing rules and see all the possible ways we could have derived the fact that $e : \tau$, that is, that $\bar{n}_1 + \bar{n}_2 : \tau$. Technically, we then have a case for each rule that could be used to prove this fact. In this case, there's only one, TYPEADD, so we can assume that this is the bottom rule in the derivation of $e : \tau$. Because we know this is the rule that was used, we know that $\tau = \text{int}$ (we also get that $\bar{n}_1 : \text{int}$ and $\bar{n}_2 : \text{int}$, but we knew that already).

- STEPSEARCHCATRIGHT. Basically the same as the case for STEPSEARCHADDRIGHT.
- STEPSEARCHLEN. Then $e = |e_0|$ and $e' = |e'_0|$ and $e_0 \mapsto e'_0$. By inversion on TYPELEN, $\tau = \text{int}$ and $e_0 : \text{string}$. By induction, $e'_0 : \text{string}$. By TYPELEN, $e' : \text{int}$.

□

We need the following lemma to prove Progress:

Lemma 1. *Canonical Forms*

1. If $e \text{ val}$ and $e : \text{int}$, then $e = \bar{n}$ for some n .
2. If $e \text{ val}$ and $e : \text{string}$, then $e = "s"$ for some s .

Proof. 1. By “induction”² on the derivation of $e \text{ val}$.

- VALNUM. Then $e = \bar{n}$.
- VALSTRING. Doesn’t apply because this would mean $e = "s"$ and that doesn’t have type int .

2. Similar.

□

In general, for any language we make, we’ll have a case of Canonical Forms stating what the value forms (canonical forms) of that type are.

Theorem 2 (Progress). *If $e : \tau$, then $e \text{ val}$ or there exists e' such that $e \mapsto e'$.*

Proof. By induction on the derivation of $e : \tau$.

- TYPENUM. Then $e = \bar{n}$. By VALNUM, $e \text{ val}$.
- TYPESTRING. Then $e = "s"$. By VALSTRING, $e \text{ val}$.
- TYPEADD. Then $e = e_1 + e_2$ and $\tau = \text{int}$ and $e_1 : \text{int}$ and $e_2 : \text{int}$. By induction, $e_1 \text{ val}$ or $e_1 \mapsto e'_1$ for some e'_1 .
 - Suppose $e_1 \text{ val}$. Then, by canonical forms, $e_1 = \bar{n}_1$ for some n_1 . By the other induction hypothesis, $e_2 \text{ val}$ or $e_2 \mapsto e'_2$ for some e'_2 .
 - * Suppose $e_2 \text{ val}$. Then, by canonical forms, $e_2 = \bar{n}_2$ for some n_2 . So we have $e = \bar{n}_1 + \bar{n}_2$. By STEPADD, $e \mapsto \overline{n_1 + n_2}$.
 - * Suppose $e_2 \mapsto e'_2$. Then, by STEPSEARCHADDRIGHT, $e \mapsto \bar{n}_1 + e'_2$.
 - Suppose $e_1 \mapsto e'_1$. Then, by STEPSEARCHADDLEFT, $e \mapsto e'_1 + e_2$.
- TYPECAT. Similar to TYPEADD.
- TYPELEN. Then $e = |e_0|$ and $\tau = \text{int}$ and $e_0 : \text{string}$. By induction, $e_0 \text{ val}$ or $e_0 \mapsto e'_0$ for some e'_0 .
 - $e_0 \text{ val}$. Then, by canonical forms, $e_0 = "s"$ for some s and $e = |"s"|$. By STEPLEN, $e \mapsto \overline{|s|}$.
 - $e_0 \mapsto e'_0$. Then, by STEPSEARCHLEN, $e \mapsto |e'_0|$.

□

²There are only axioms, so there’s never an induction hypothesis.